TRIPORY

Technology Stack & System Architecture

Revised technical baseline for a local-first, mobile-only travel companion.

Document	Status
Product	Tripory
Architecture	Local-first / offline-capable
Platforms	Android + iOS
Cloud role	Optional services + manual Google Drive backup
Dashboard	Out of scope for current phase
Revision	Repository structure and presentation refined

Core decision

The device is the primary home of Tripory data. Internet is a capability dependency for maps, place search, some GPS enrichment, Google authentication and manual backup—not for the core trip workflow.

01. Executive Summary

Tripory should be built as a local-first mobile application, rather than a conventional cloud-first SaaS product. Core trip data lives on the device and remains usable when the user is offline. Online services are introduced only where they provide external capabilities.

This decision intentionally removes a custom backend and cloud database from the MVP. The engineering focus shifts toward local database integrity, media storage, backup/restore reliability, security and Android/iOS consistency.

Recommended baseline

- React Native + Expo + TypeScript — shared Android/iOS codebase.
- SQLite + Drizzle ORM — relational local persistence.
- Device filesystem — photos and other binary media.
- Zustand — transient/application state, not the persistence layer.
- Google OAuth + Google Drive API — optional account connection and manual backup/restore.
- Google Maps or Mapbox — map visualization and location-related capabilities.
- No custom backend / cloud DB in MVP.

Platform correction: Android uses APK/AAB. iOS uses IPA internally and App Store/TestFlight distribution. DMG is for macOS and is not a target for the current mobile product.

02. Product & Technical Requirements

Requirement	Implication	Priority
Local-first	Create/edit/view core trip data without internet.	Critical
Android + iOS	One cross-platform codebase preferred.	Critical
Mobile only	No web dashboard in current scope.	High
Manual backup	User explicitly uploads a backup to Google Drive.	High
Manual restore	User can select and restore a backup.	High
Maps / GPS	External network may be needed for map/search enrichment.	High
Memory / media	Photos remain locally available.	High
Privacy	Travel, location, notes and photos are sensitive user data.	Critical

Terminology

- Backup = copy a known local state to a cloud destination.
- Restore = import a selected backup into the local app.
- Sync = continuous bidirectional reconciliation; explicitly out of scope for MVP.

03. Architecture Principles

Local is the source of truth

SQLite and local media are authoritative for day-to-day use.

Offline by default

No blocking network dependency for core trip workflows.

External services are adapters

Maps, Drive and authentication sit at the edge of the system.

Persisted data ≠ UI state

SQLite owns persistent entities; Zustand owns transient application state.

Backup is explicit

Do not silently upload personal travel data in MVP.

Fail gracefully

An unavailable map must not make a trip unavailable.

Prefer managed infrastructure

Do not operate servers until a real product requirement demands them.

Design for migration

Database schemas and backup formats must evolve safely.

04. System Architecture

The proposed architecture is intentionally compact:

Layer	Responsibility	Technology
Presentation	Screens, components, forms and navigation.	React Native + Expo
Domain	Trip, itinerary, packing, memory and backup rules.	TypeScript modules
Persistence	Entities, migrations and repositories.	SQLite + Drizzle
Device	Camera, filesystem, GPS and notifications.	Expo / native APIs
External	Auth, Drive, maps and places.	Provider adapters

Logical dependency

UI → Domain Services → Repository / Provider Adapter → SQLite / Device / External API

Screens should not contain SQL, direct filesystem orchestration or Google Drive SDK calls. This boundary is important for testing and future provider replacement.

No backend in MVP

There is deliberately no Node/NestJS service, PostgreSQL server, Redis layer or cloud database in the first architecture. If later requirements introduce multi-device sync, collaboration or remote processing, a backend can be added then.

05. Recommended Technology Stack

Layer	Choice	Why
Mobile	React Native + Expo	One shared Android/iOS codebase with device capability access.
Language	TypeScript	Strong typing and maintainability.
Navigation	Expo Router	Structured navigation and deep-link support.
Local database	SQLite	Mature relational local persistence.
DB layer	Drizzle ORM	Typed schema, queries and migrations.
State	Zustand	Simple application/UI state.
Media	Device filesystem	Keeps large binary assets out of SQLite.
Authentication	Google OAuth / Sign-In	Only needed when connecting Google services.
Backup	Google Drive API	User-controlled cloud destination.
Location	Expo Location / native APIs	Device location access.
Maps	Google Maps or Mapbox	Map display and place features.
Notifications	Expo Notifications / native APIs	Reminders and notifications.
Build	EAS Build	Android/iOS build pipeline.
Source control	GitHub	Repository, PRs, issues and CI.
CI	GitHub Actions	Lint, typecheck and automated tests.
Monitoring	Sentry — later	Crash/non-fatal error visibility.
Analytics	Optional	Only collect what is needed for product decisions.

06. Mobile Application Architecture

Organize the code by product domains rather than only by generic UI categories.

Suggested high-level structure

PATH	PURPOSE
`app/`	Expo Router routes and screen entry points.
`src/core/`	Shared utilities, constants and cross-cutting primitives.
`src/db/`	SQLite schema, migrations and repository implementations.
`src/domains/`	Trip, itinerary, packing, activity, memory and backup logic.
`src/services/`	Location, maps, Google, notifications and provider adapters.
`src/components/`	Reusable presentation components.
`src/stores/`	Zustand stores for transient application state.
`src/hooks/`	Reusable React hooks.
`src/types/`	Shared TypeScript types.
`tests/`	Unit, integration and E2E test suites.
`docs/`	ADRs, technical notes and implementation documentation.

State ownership

- SQLite: trips, itinerary, packing items, activities, moments, media metadata and backup records.
- Zustand: selected trip, UI state, operation progress and temporary session state.
- Component state: form fields and ephemeral interaction.

This prevents a common failure mode: duplicating the same truth across local DB, global state and UI state.

07. Local Data Architecture

Entity	Purpose
trips	Trip identity, title, destination, dates, status and timestamps.
itineraries	Schedule entries, notes, ordering and optional locations.
packing_items	Item, category, quantity, packed/used/returned state.
activities	Activities, status and optional location/time.
moments	Memory content, caption, timestamp and location.
media_assets	Local path, MIME type, size, dimensions and checksum/hash.
locations	Coordinates, labels and provider identifiers.
backup_records	Backup version, date, destination and manifest reference.
app_metadata	Schema version, migration state and app-level metadata.

Relationship model

Trip → Itinerary / Packing Item / Activity / Moment → Media Asset / Location. Binary media is stored separately; SQLite stores references and metadata.

Migration strategy

- Every schema change has a migration.
- Backup includes schema/app compatibility metadata.
- Restore validates compatibility before changing local data.
- Destructive restore should create a safety backup first when possible.

08. Media & File Storage

Tripory's memory feature makes media architecture critical. Photos should not be stored as large blobs inside SQLite.

Concern	Recommendation
Binary media	Application-controlled device filesystem.
Database reference	Stable media ID + relative path + metadata.
Naming	Stable IDs rather than filenames as identifiers.
Deletion	Controlled cleanup to avoid orphaned files/rows.
Backup	Include all referenced media.
Integrity	Use size/checksum information for validation.
Storage growth	Track media size; later add compression/cleanup/export.

Media lifecycle

> Capture/import → temporary file → validate → move to stable path → create media row → associate with moment/trip → optionally include in backup.

A database row must not be treated as proof that a file still exists. Validate file existence at important boundaries.

09. Google Account & Backup / Restore

The requested 'Gmail sync' should technically be implemented as Google Account authentication + Google Drive storage. Gmail is the email service; Drive is the appropriate backup destination.

Backup flow

User selects Backup → Google authorization → generate package → validate manifest → upload to Drive → record backup locally.

Restore flow

User selects Restore → authenticate → select backup → download → validate version/checksums → create safety backup → restore → rebuild indexes/cache → verify.

Package	Purpose
manifest.json	Format version, app version, timestamp, file list and checksums.
database	SQLite export or logical data export.
media/	Photos and future media.
metadata/	Optional recovery/debug metadata.

Backup rules

- Manual and user-visible in MVP.
- Versioned backup format.
- Validation before restore.
- Dedicated Tripory folder in Drive.
- Incremental backup is a later optimization, not an MVP prerequisite.

Backup is not sync. Multi-device sync requires identity, remote revisions, conflict resolution, deletions/tombstones, media synchronization and a remote source of truth.

10. Maps, GPS & External Services

Capability	Type	Role
GPS / location	Device	Capture coordinates with permission.
Map rendering	External	Display maps, pins and routes.
Place search	External	Search destinations and locations.
Geocoding	External	Translate coordinates and place labels.
Google Drive	External	Backup and restore.

Provider abstraction

Create interfaces such as LocationService, PlaceSearchService, MapService and BackupProvider. The domain depends on these interfaces, not directly on vendor SDKs.

This prevents a future provider migration from becoming a product rewrite.

11. Offline-First & Data Availability

Operation	Mode	Expected behavior
Create/edit trip	Offline	Always.
Packing/checklist	Offline	Always.
Notes/memories	Offline	Always.
View memories	Offline	Local media permitting.
Capture photo	Offline	Permission/storage permitting.
GPS capture	Offline-capable	Depends on device location.
Maps/place search	Online-dependent	Show graceful unavailable state.
Google login	Online	Only when connecting account.
Backup/restore	Online	Requires Drive access.

Offline UX rule

Never replace the whole application with a 'No Internet' screen. Only the unavailable capability should be blocked—for example, 'Map unavailable offline' while the itinerary remains usable.

12. Security & Privacy

Local-first reduces server exposure but does not automatically make the application secure. Travel dates, locations, notes and photos should be treated as private user data.

Baseline controls

- Use platform-secure storage for OAuth tokens; never store tokens in plain SQLite.
- Request only permissions required for the active feature.
- Do not embed private server secrets in the app.
- Treat backup packages as sensitive data.
- Use HTTPS/TLS for network traffic.
- Validate backup versions, manifests, file types, sizes and checksums.
- Prevent arbitrary file writes/path traversal during restore.
- Use least-privilege Google Drive scopes.
- Do not collect detailed location/travel content through analytics.
- Provide clear local-data deletion and explain what remains in Drive.

Encryption

Application-level encryption for backups should be evaluated against a defined threat model. It can add protection, but it also creates key/password recovery complexity. Do not introduce custom cryptography casually.

13. Build, Release & Distribution

Platform	Target
Android	AAB for Play Store; APK for direct/internal testing where appropriate.
iOS	IPA is the package; distribute through TestFlight/App Store.
macOS	Out of scope; DMG is a macOS installer format.

Environments

- Development: emulator/simulator + real devices.
- Internal testing: development builds.
- Staging: production-like configuration with controlled integrations.
- Production: signed store builds.

Separate environment variables, OAuth client IDs, map keys and other configuration. Never commit secrets to Git.

14. Testing Strategy

Level	Focus
Unit	Domain rules, validation, dates, packing state, backup manifest.
Repository	SQLite CRUD, migrations and referential integrity.
Integration	Create → persist → reopen; media; backup/restore.
Provider	Google auth, Drive, map/location adapters.
Offline	All core workflows with network disabled.
Failure recovery	Interrupted backup, corrupt file, missing media, invalid restore.
Cross-platform	Android/iOS filesystem, camera, location, notifications, OAuth.
E2E	Critical user journeys on real devices.

Highest-risk scenarios

- Kill the app during backup.
- Restore a backup over newer local data.
- Restore with missing/corrupt media.
- Run all core workflows in airplane mode.
- Reinstall and restore from Drive.
- Migrate an older database schema.

15. Observability & Analytics

A local-first app has less server-side telemetry. Diagnostics should therefore focus on crashes, backup failures and provider integration errors.

Recommended after core stability

- Sentry for crash and non-fatal error reporting.
- Structured error categories: DB, filesystem, backup, OAuth, maps/location and notifications.
- Bounded diagnostic logs; never log private trip content unnecessarily.
- Optional analytics only when it supports a real product decision.

Metric	Purpose
Crash-free sessions	Application stability.
Backup success rate	Reliability of the primary safety mechanism.
Restore success rate	Backup integrity.
Offline core-flow success	Whether local-first behavior works.
Media import failure rate	Health of the memory feature.

16. Development Environment & Repository

A single repository is appropriate for the current mobile-only product. The structure below is intentionally explicit so developers can understand where each responsibility belongs without relying on a long wrapped directory string.

Repository structure

PATH	PURPOSE
`tripory/`	Project root.
`├── app/`	Expo Router screens, routes and navigation entry points.
`├── src/`	Application source code.
`│ ├── core/`	Shared utilities, constants and cross-cutting primitives.
`│ ├── db/`	SQLite schema, migrations and repositories.
`│ ├── domains/`	Trip, itinerary, packing, activity, memory and backup logic.
`│ ├── services/`	Location, maps, Google, notifications and provider adapters.
`│ ├── components/`	Reusable UI/presentation components.
`│ ├── stores/`	Zustand stores for transient application state.
`│ ├── hooks/`	Reusable React hooks.
`│ └── types/`	Shared TypeScript types.
`├── assets/`	Icons, images, fonts and static assets.
`├── tests/`	Unit, integration and E2E tests.
`├── docs/`	ADRs, architecture notes and technical documentation.
`├── app.config.*`	Expo application configuration.
`├── eas.json`	EAS Build profiles and release configuration.
`├── package.json`	Dependencies and project scripts.
`└── README.md`	Developer setup, commands and project overview.

Quality baseline

Area	Baseline
Type safety	TypeScript strict mode.
Code quality	Linting + formatting enforced in CI.
CI	Lint, typecheck and tests on pull requests.
Version control	Git + pull requests for meaningful changes.
Database	Every schema change is a reviewed migration.
Architecture	Major decisions documented as ADRs.
Environment	Secrets and provider configuration separated by environment.

Why this revision	The repository tree is now represented as a structured table instead of one long text line. This prevents wrapping/overlap and makes the ownership of each folder immediately readable.

17. MVP Scope vs Future Architecture

Horizon	Direction
MVP	Local SQLite, local media, manual Drive backup/restore, maps/GPS and Google account connection when needed.
Near future	Incremental backups, encrypted backup packages, media compression, richer notifications and optional offline map caching.
Later	Cross-device sync, shared trips, web dashboard, collaboration and server-side services.

If sync is introduced later

Do not bolt sync onto the MVP as simple upload/download. Introduce explicit identity, remote revisions, conflict policy, deletion/tombstones, media sync, retry queues and server authorization. Only then is a backend/cloud database justified.

18. Risks, Trade-offs & Decisions

Risk	Severity	Mitigation
Local data loss	High	Make backup discoverable; show last-backup state.
Device storage growth	High	Track media size; add cleanup/compression/export.
Restore corruption	High	Manifests, checksums, versioning and safety backup.
Provider dependency	Medium	Adapter interfaces around maps and backup.
OAuth complexity	Medium	Isolate Google integration; keep core app usable without account.
Android/iOS differences	Medium	Test real devices for filesystem, permissions, notifications and OAuth.
Future sync pressure	Medium	Use stable IDs and clean domain boundaries.

Decisions to lock before implementation

- React Native + Expo vs Flutter vs bare React Native.
- SQLite implementation and ORM choice.
- Google Drive backup package and encryption strategy.
- Maps provider and cost/licensing assumptions.
- Google OAuth scopes and Drive folder strategy.
- Media retention/compression policy.
- Minimum supported Android/iOS versions.

19. Implementation Roadmap

Phase	Focus	Output
01	Foundation	React Native + Expo, TypeScript, navigation, config, linting and CI.
02	Persistence	SQLite + Drizzle schema, migrations, repositories and trip CRUD.
03	Core domains	Trip, itinerary, packing and activity models/screens.
04	Media	Filesystem media, photo import/capture, metadata and cleanup.
05	Location	GPS, permissions, map adapter and place lookup.
06	Memory	Moments, captions, locations and album retrieval.
07	Google	OAuth and Drive integration behind BackupProvider.
08	Backup	Versioned package, upload, list, download, validation and restore.
09	Hardening	Offline tests, recovery, migrations, security and performance.
10	Release	Android/iOS internal builds, store testing and production release.

First technical spike

> Create trip → persist SQLite → attach photo → close/reopen app → connect Google → upload backup → delete local data → restore backup → verify trip + photo. Run this on both Android and iOS before building the full product.

If this works reliably, the core architecture is validated. If it fails, the problem is discovered before most UI work is built.

Appendix — Architecture Decision Records

ADR	Decision	Statement
ADR-001	Local-first	Device storage is the source of truth for core trip data.
ADR-002	Mobile-only MVP	No dashboard/web app in current scope.
ADR-003	Cross-platform	React Native + Expo is the default unless the technical spike disproves it.
ADR-004	SQLite	Use relational local persistence for core entities.
ADR-005	Media outside DB	Store binary media in filesystem; references in SQLite.
ADR-006	Manual backup	Google Drive backup/restore is explicit; continuous sync is out of scope.
ADR-007	No custom backend	Do not operate backend infrastructure until a concrete requirement justifies it.

Final architectural position

Tripory should be a local-first, cross-platform mobile application with relational local persistence, device-managed media and isolated online integrations. The absence of a backend in MVP is intentional: it keeps technology aligned with actual product requirements instead of introducing infrastructure before it creates user value.

This document is a technical baseline. Framework/provider decisions should be validated by the technical spike and captured as ADRs before production implementation.